

EXAMEN STRUCTURES DE DONNÉES AVANCÉES ET PROGRAMMATION

(Documents non autorisés)

Exercice 1 (10 points)

Soit la déclaration des types suivants :

```
struct Date {
    int jour, mois, annee;
};
typedef struct Date Date;

struct Produit {
    char ident[20], lib[20];
    float pu;
};
typedef struct Produit Produit;

struct Commande {
    char ident[20];
    Date dateC;
    Produit *prod;
    int nbProd;
};
typedef struct Commande Commande;

struct Client {
    char nom[20], prenom[20], tel[20];
    Date ddn;
    Commande *cmd;
    int nbCom;
};
typedef struct Client Client;
```

1. Ecrire une fonction `lireClient()` qui lit un client et le renvoie. (5 points)
2. Donner le code de la fonction `afficher(Client cli)` qui permet d'afficher les informations de la variable `cli` paramètre de la fonction. (5 points)

Exercice 2 : Occupation de la mémoire (10 points)

Il est demandé dans cet exercice de représenter en mémoire les données déclarées dans un programme, ainsi que leurs différentes valeurs, à un moment donné de l'exécution. Pour cela, vous représenterez l'occupation des données en mémoire dans un tableau à 3 colonnes (Identificateur, Adresse, Valeur).

Pour déterminer les adresses, on fera les approximations suivantes :

- Les données sont réservées en mémoire dans l'ordre de leur déclaration.
- La première adresse démarre à 1000 (en notation décimale pour faciliter les calculs).
- L'architecture est 32 bits (un entier `int` occupe 4 octets, un pointeur occupe 4 octets).

Le programme est donné ci-dessous. Notez que le programme fait une utilisation abusive des pointeurs, l'objectif étant simplement de vous familiariser avec la syntaxe et la sémantique des instructions manipulant des pointeurs.

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    int a = 10;
    int b = 5;
    int tab[3] = {1, 2, 3};
    int *p_int;

    /* Question 1 : Reproduire et compléter le tableau ci-dessous (4 points) */

    /* Suite du programme */
    tab[0] = a;
    *(tab + 1) = b;
    p_int = tab + 2;

    /* Question 2 : Reproduire et compléter le tableau après exécution (3 points) */
    return 0;
}
```

Tableau Question 1 :

Identificateur	Adresse	Valeur
a	1000	
b		
tab[0]		
tab[1]		

Identificateur	Adresse	Valeur
tab[2]		
tab		

Tableau Question 2 :

Identificateur	Adresse	Valeur
a		
b		
tab[0]		
tab[1]		
tab[2]		
tab		
p_int		

STRUCTURES DE DONNÉES AVANCÉES ET PROGRAMMATION

(Documents non autorisés)

Exercice 1 (6 points)

Quels résultats fournira ce programme ?

```
main() {  
    int t[3];  
    int i, j;  
    int *adt;  
  
    for (i = 0, j = 0; i < 3; i++) {  
        t[i] = ++j + i;  
    }  
}
```

1.

```
for (i = 0; i < 3; i++)  
    printf("%d ", t[i]);
```
2.

```
for (i = 0; i < 3; i++)  
    printf("%d ", *(t + i));
```
3.

```
for (adt = t; adt < t + 3; adt++)  
    printf("%d ", *adt);
```
4.

```
for (adt = t + 2; adt >= t; adt--)  
    printf("%d ", *adt);
```

Exercice 2 (6 points)

La conjecture de Goldbach est formulée en 1742 par Christian Goldbach, c'est l'un des plus vieux problèmes non résolus de la théorie des nombres.

Énoncé : Tout nombre entier pair supérieur à 3 peut s'écrire comme la somme de deux nombres premiers.

EXAMEN DE PROGRAMMATION

Exercice 1 : Questions de cours (6 points)

a. Qu'affiche le programme suivant ? (2 points)

```
main() {  
    int a[5] = {5, 1, 15, 20, 25};  
    int i, j, k = 1, m;  
  
    i = ++a[1];  
    j = a[1]++;  
    m = a[i++];  
  
    printf("  
%d %d %d", i, j, m);  
}
```

b. Écrire avec une boucle `while` l'instruction suivante (1 point) :

```
for (A; B; C) { D; }
```

c. On définit les variables de la façon suivante (3 points) :

```
int i;  
char c; int *pti;  
char *ptc;
```

Choisir ce qui est juste :

1. `pti = &i; *pti = 12;`
2. `pti = &tab; *pti = 4;`
3. `ptc = &c; *ptc = 'a';`
4. `tab[i]` est équivalent à `*(tab + i)`
5. `tab[i]` est équivalent à `&(tab + i)`

Exercice 2 (8 points)

Soit une matrice d'entiers.

1. Écrire une fonction qui permet d'initialiser la matrice avec des valeurs décroissantes aléatoires. (3 points)
2. Si la matrice est constituée de N lignes et de M colonnes, écrire une fonction qui permet d'afficher ses valeurs sur N lignes et M colonnes. (2 points)
3. Écrire une fonction qui permet de substituer un élément de la matrice par une valeur transmise en argument. La position de cet élément est aussi donnée en argument. (3 points)

Exercice 3 (6 points)

Écrire un programme C qui définit une structure permettant de stocker le nom, le prénom et l'âge d'une personne. Le programme initialise ensuite deux personnes et affiche le nom complet de la moins âgée d'entre elles en utilisant une seule fonction `printf` pour l'affichage du résultat.

Exercice 1 (4 points)

Qu'affiche le programme suivant :

```
#include <stdio.h>
#define taille_max 5

void parcours(int *tab) {
    int *q = tab;
    do {
        printf("%d:%d", *q - tab, *q - *tab);
    } while (++q - tab < taille_max);
}

int main() {
    int chose[taille_max] = {1, 3, 2, 4, 5};
    printf("chose : ");
    parcours(chose);
    return 0;
}
```

Exercice 2 (12 points)

1. Définir une structure `Produit` caractérisée par les champs `Code`, `Nom` et `PrixUnitaire`. (2 points)
2. Créer une fonction `LireProduit` qui demande à l'utilisateur les informations d'un enregistrement de type produit pour le lire et le retourner. (3 points)
3. Créer une fonction `AfficherProduit` qui prend un tableau de produits en paramètre et l'affiche à l'écran. (3 points)
4. Pour exploiter ces fonctions et structure, écrire un programme principal qui affiche à l'utilisateur le menu suivant et réalise l'opération demandée par l'utilisateur en utilisant les fonctions définies précédemment : (4 points)
 - 1: Lire les produits
 - 2: Afficher la liste des produits
 - 3: Quitter le programme

Exercice 3 (4 points)

Écrire une fonction en langage C qui prend un entier en paramètre et retourne 1 si cet entier est parfait et 0 sinon. Un nombre parfait est un entier qui est égal à la somme de ses diviseurs stricts (Exemple : 6 est parfait car $6 = 1 + 2 + 3$).

Examen du Premier Semestre (Session 1)
Structures De Données Avancées Et Programmation

Durée : 2h • Documents non autorisés

Exercice 1 : Syntaxe de base (4 points)

Soit le programme ci-après :

```
#include <stdio.h>
int main() {
    int i, n, s;
    printf("Donner un entier positif : ");
    scanf("%d", &n);
    for (i = s = 1; i <= n; s *= i++);
    printf("Le resultat est %d", s);
}
```

1. Décomposer l'expression `s *= i++;` (1,5 pts)
2. Que fait le programme ? (1 pt)
3. Donner la sortie pour `n = 5`. (1,5 pts)

Exercice 2 : Enregistrement (5 points)

Dans cet exercice, l'objectif est de proposer un programme de gestion des clients d'une banque de la place. Le client est caractérisé par son nom, son prénom, sa date de naissance, son numéro CIN, le nombre de comptes et les informations des comptes bancaires (penser aux pointeurs).

- Un compte bancaire est caractérisé par un numéro, un montant, type de compte.
- Une date de naissance est caractérisée par le jour, le mois et l'année.

1. Créer une structure `Client` en utilisant les structures `Compte` et `Date`. (2 pts)
2. Ecrire un programme permettant de saisir et d'afficher toutes les informations d'un client. (3 pts)

Exercice 3 : Tableaux (5 points)

Soit une personne caractérisée par son nom, prénom et âge ; après avoir créé la structure `personne` et déclaré un tableau de 100 personnes :

1. Proposer un programme permettant de lire un tableau de `n` personnes (`n` une valeur à saisir qui doit être entre 0 et 100 représentant la taille effective du tableau). (2,5 pts)
2. Dans la suite, proposer un programme permettant d'insérer une personne lue à une position donnée (entre 1 et `n+1`). (2,5 pts)

Exercice 4 : Pointeurs et tableaux (6 points)

Soit `ptr` un pointeur qui "pointe" sur un tableau `tab` :

```
int tab[] = {12, 23, 34, 45, 56, 67, 78, 89, 90};  
int *ptr = NULL;  
ptr = tab;
```

Quelles valeurs ou adresses fournissent ces expressions :

- A. `*ptr + 2;`
- B. `*(ptr + 2);`
- C. `&tab[0] + 1;`
- D. `tab + 3;`
- E. `ptr + (*ptr - 10);`
- F. `*(ptr + *(ptr + 8) - tab[7])`

Barème de notation : Bonne réponse = 1 pt / Mauvaise réponse = -0,5 pt / Absence de réponse = 0 pt